# Advanced Terraform on AWS
# Lab 1 – Churn

Expected Duration ..... 1
Teaching Points – What This Lab Is Designed to Teach ..... 1
Before you begin ..... 2
Phase 1 – Initial deployment ..... 2
Phase 2 – Configuration churn due to mutable attribute change ..... 3
Phase 3 – Identity churn due to immutable attribute change ..... 6
Phase 4 – Stable keys with for_each ..... 8
Phase 5 – Clean up ..... 10

## Expected Duration

This lab should take 45–60 minutes to complete.

## Teaching Points – What This Lab Is Designed to Teach

This lab is deliberately not about building a sophisticated AWS network security design. It is about understanding how Terraform state, resource identity, and input data structures interact over time.

Across the phases in this lab, you will create and modify security groups inside a simple VPC. The infrastructure itself is straightforward. The learning value comes from observing how Terraform reacts as the variable structure becomes richer and the resource addressing model changes.

By the end of the lab, you should be able to clearly explain:

- Why Terraform sometimes proposes large numbers of changes even when your intent feels minimal
- Why simple ordered lists are fragile foundations for long-lived infrastructure
- How count ties resource identity to ordinal position in state
- How for_each changes resource addressing from positional indexes to stable keys
- Why map(list) with for_each is a more stable design than list-based iteration

This lab progresses intentionally:

- Early phases create instability so you can observe it.
- Middle phases explain why it happens.
- Final phases remove the causes, not just the symptoms.

The goal is not simply to reach a clean end state, but to understand why that end state is stable and why the earlier designs were not.

## Before you begin

1. Navigate to the working directory **c:\qatfadvaws\lab1\guided** which contains main.tf, variables.tf, outputs.tf, and terraform.tfvars.
2. Open a terminal in this directory.
3. This will be your working directory and root module for the lab. Make all changes in this directory only. Directory **c:\qatfadvaws\lab1\original** contains a copy of the files used in this lab and are for reference/reset purposes if needed.

## Phase 1 – Initial deployment

4. Review main.tf, variables.tf, outputs.tf, and terraform.tfvars...

   This configuration creates a VPC, a subnet, and a set of AWS security groups. At this stage, the security groups are driven from a simple list of strings and are created using count.

   As shown in terraform.tfvars, there are currently two elements defined in the security_groups variable:

   ```
   web
   app
   ```

   Terraform will therefore create two security group resources using the count index for addressing.

   Expected state addresses:

   ```
   aws_security_group.demo[0]
   aws_security_group.demo[1]
   ```

   Expected security group names:

   ```
   secgp-0
   secgp-1
   ```

   Expected Name tags:

   ```
   secgp-web
   secgp-app
   ```

5. Deploy the resources:

   ```
   terraform init
   terraform apply --auto-approve
   ```

6. Verify the resources that are created in state:

```
terraform state list
```

You should see entries similar to the following:

```
aws_vpc.main
aws_subnet.demo
aws_security_group.demo[0]
aws_security_group.demo[1]
```

```
terraform state show aws_security_group.demo[0]
```

Observe carefully what Terraform is actually tracking...

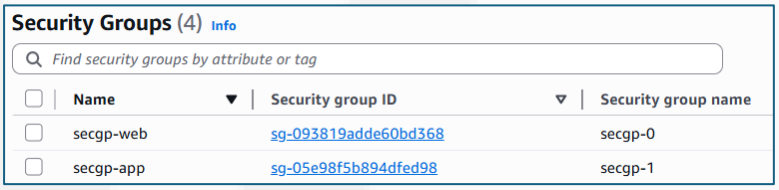
```
# aws_security_group.demo[0]:
resource "aws_security_group" "demo" {
    arn                    = "arn:aws:ec2
```

The state resource address is aws_security_group.demo[0]. That means Terraform is currently identifying the resource by list position, not by its logical meaning.

Notice that the security group name is derived from the list index (0) and the Name tag is derived from the value at that index position (web)...

```
name                   = "secgp-0"
name_prefix            = null
owner_id               = "503561439433"
revoke_rules_on_delete = false
tags                   = {
    "Name" = "secgp-web"
    "Role" = "web"
```

7. Verify the new resources in the AWS portal ...



Security Groups (4) Info

Find security groups by attribute or tag

| Name | Security group ID | Security group name |
|---|---|---|
| secgp-web | sg-093819adde60bd368 | secgp-0 |
| secgp-app | sg-05e98f5b894dfed98 | secgp-1 |

**Important**: In AWS, the 'Name' of a security group is an optional tag and is mutable, whereas the 'Security group name' is mandatory and immutable.

# Phase 2 – Configuration churn due to mutable attribute change

## Ordinal stability

8. In **terraform.tfvars,** comment line 10 and uncomment line 11 so that **db** is added at the end of the list.

9. Plan the changes:

```
terraform plan
```

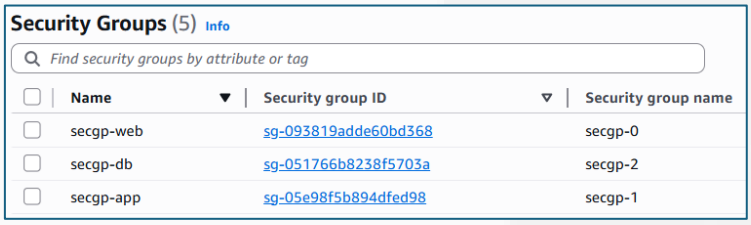
The plan should show 1 to add, 0 to change, 0 to destroy.

The new resource will be aws_security_group.demo[2]. This will have no effect on the existing resources [0] and [1], because their positions in the list have not changed.

10. Apply the changes:

```
terraform apply --auto-approve
```

11. Verify the new resource in the AWS portal ...



Security Groups (5) Info

Find security groups by attribute or tag

| Name | Security group ID | Security group name |
|---|---|---|
| secgp-web | sg-093819adde60bd368 | secgp-0 |
| secgp-db | sg-051766b8238f5703a | secgp-2 |
| secgp-app | sg-05e98f5b894dfed98 | secgp-1 |

When using count, adding items to the end of a list usually produces stable behaviour. Terraform adds the new resource and leaves the existing indexed resources untouched.

## Ordinal instability

12. In **terraform.tfvars**, comment line 11 and uncomment line 12 so that **admin** is inserted at the start of the list.
13. Before running Terraform, stop and consider what you expect to happen. You have added one new security group name, but you have inserted it at the start rather than the end.
14. Plan the changes:

```
terraform plan
```

Review the plan carefully, is should show 1 to add, 3 to change, 0 to destroy.

Terraform is still using count, so the resources are still addressed as [0], [1], [2] and so on. However, the values attached to those positions have shifted.

That means:

demo[0] (secgp-0) Name **secgp-web** changes to **secgp-admin**
demo[1] (secgp-1) Name **secgp-app** changes to **secgp-web**
demo[2] (secgp-2) Name **secgp-db** changes to **secgp-app**
demo[3] (secgp-3) is created with Name **secgp-db**

15. Apply the changes:

```
terraform apply --auto-approve
```

16. Verify the updated resource in the AWS portal ...

**Security Groups** (6) Info

Find security groups by attribute or tag

| Name | Security group ID | Security group name |
|---|---|---|
| secgp-db | sg-05e1c3332390498f5 | secgp-3 |
| secgp-app | sg-051766b8238f5703a | secgp-2 |
| secgp-web | sg-05e98f5b894dfed98 | secgp-1 |
| secgp-admin | sg-093819adde60bd368 | secgp-0 |

17. In **terraform.tfvars**, comment line 12 and uncomment line 13 so that **web** is removed from the list.

    Before running Terraform, stop and consider what you expect to happen. The Name attribute for security groups [1] and [2] have changed and security group [3] is no longer needed. The mutable attribute changes will therefore triggers change actions for security groups [1] and [2] along with the destruction of security group [3].

18. Plan the changes:

    ```
    terraform plan
    ```

    Review the plan carefully, it should show 0 to add, 2 to change, 1 to destroy.

    Terraform is still using count, so the resources are still addressed as [0], [1], [2] and so on. However, the values attached to those positions have shifted.

    That means:

    demo[0] (secgp-0) Name **secgp-admin** remains unchanged
    demo[1] (secgp-1) Name **secgp-web** changes to **secgp-app**
    demo[2] (secgp-2) Name **secgp-app** changes to **secgp-db**
    demo[3] (secgp-3) is destroyed

19. Apply the changes:

    ```
    terraform apply --auto-approve
    ```

20. Verify the new resources in the AWS portal ...

**Security Groups** (5) Info

Find security groups by attribute or tag

| Name | Security group ID | Security group name |
|---|---|---|
| secgp-db | sg-051766b8238f5703a | secgp-2 |
| secgp-app | sg-05e98f5b894dfed98 | secgp-1 |
| secgp-admin | sg-093819adde60bd368 | secgp-0 |

    **What this demonstrates:** What looks like a small logical change can produce a much larger operational effect. A simple ordered list is fragile. Even when your intent is small, ordinal indexing can cause widespread change because resource configuration is tied to position.

21. In **terraform.tfvars**, comment line 13 and uncomment line 14 so the list is empty. This will remove all existing security groups ahead of the next phase.

22. Apply the changes:

    ```
    terraform apply --auto-approve
    ```

## Phase 3 – Identity churn due to immutable attribute change

23. In **main.tf**, comment line 43 and uncomment line 44. This binds the 'Security group name' to be given to the security group to the explicit positional value in the security_groups variables. A positional change will invoke a 'Security group name' change, an immutable attribute in AWS.

24. In **terraform.tfvars**, comment line 14 and uncomment line 10. This adds 2 elements to the security_groups list.

25. Plan the changes:

    ```
    terraform plan
    ```

    Review the state addresses again after apply:

    ```
    terraform apply --auto-approve
    terraform state list
    ```

    You should still see addresses such as:

    ```
    aws_security_group.demo[0]
    aws_security_group.demo[1]
    ```

    ```
    terraform state show aws_security_group.demo[0]
    ```

    ```
    name                   = "secgp-web"
    name_prefix            = null
    owner_id               = "503561439433"
    revoke_rules_on_delete = false
    tags                   = {
        "Name" = "secgp-web"
        "Role" = "web"
    ```

26. Verify the new resources in the AWS portal …

    **Security Groups** (4) Info

    Find security groups by attribute or tag

    | Name | Security group ID | Security group name |
    |---|---|---|
    | secgp-web | sg-08e6d327e252c460c | secgp-web |
    | secgp-app | sg-0f63a3dbc470fd0d0 | secgp-app |

27. In **terraform.tfvars,** comment line 10 and uncomment line 11 so that **db** is added at the end of the list.

28. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show 1 to add, 0 to change, 0 to destroy.

    The new resource will be aws_security_group.demo[2]. This will have no effect on the existing resources [0] and [1], because their positions in the list have not changed.

29. Apply the changes:

    ```
    terraform apply --auto-approve
    ```

30. Verify the updated resources in the AWS portal …

**Security Groups** (5) Info

Find security groups by attribute or tag

| Name | Security group ID | Security group name |
|---|---|---|
| secgp-web | sg-08e6d327e252c460c | secgp-web |
| secgp-db | sg-083c6e98a25a5b605 | secgp-db |
| secgp-app | sg-0f63a3dbc470fd0d0 | secgp-app |

31. As before, when using count, adding items to the end of a list usually produces stable behaviour. Terraform adds the new resource and leaves the existing indexed resources untouched.
32. In **terraform.tfvars**, comment line 11 and uncomment line 12 so that **admin** is inserted at the start of the  list.
33. Plan the changes:

    ```
    terraform plan
    ```

    Review the plan carefully, it should show 4 to add, 0 to change, 3 to destroy.

    ```
    Changes to Outputs:
      ~ security_group_addresses = [
          - "aws_security_group.demo[0] => secgp-web",
          - "aws_security_group.demo[1] => secgp-app",
          - "aws_security_group.demo[2] => secgp-db",
          + "aws_security_group.demo[0] => secgp-admin",
          + "aws_security_group.demo[1] => secgp-web",
          + "aws_security_group.demo[2] => secgp-app",
          + "aws_security_group.demo[3] => secgp-db",
    ```

    The 'Security group name' used for the security groups [0],[1] and [2] has now changed. This is an immutable attribute and therefore triggers destroy/add actions along with the creation of security group [3].

34. Apply the changes:

    ```
    terraform apply --parallelism=1 --auto-approve
    ```

35. Verify the **new** resources in the AWS portal ...

**Security Groups** (6) Info

Find security groups by attribute or tag

| Name | Security group ID | Security group name |
|---|---|---|
| secgp-web | sg-01c8d42b2b28ce5dd | secgp-web |
| secgp-db | sg-03b8ef5c988826820 | secgp-db |
| secgp-app | sg-049e8c48c51dd6a82 | secgp-app |
| secgp-admin | sg-04be5720185a18b2f | secgp-admin |

    Note the new IDs !

36. In **terraform.tfvars**, comment line 12 and uncomment line 13 so that **web** is removed from the list.

    Before running Terraform, stop and consider what you expect to happen. The 'Security group name' used for the security groups [1] and [2] have changed and security group [3] is no longer needed. The immutable attribute changes will therefore triggers destroy/add actions for security groups [1] and [2] along with the destruction of security group [3].

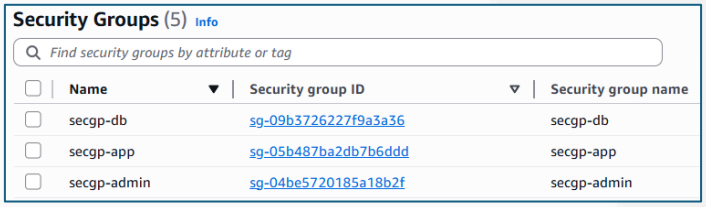
37. Plan the changes:

```
terraform plan
```

Review the plan carefully, it should show 2 to add, 0 to change, 3 to destroy

```
Changes to Outputs:
  ~ security_group_addresses = [
        "aws_security_group.demo[0] => secgp-admin",
      - "aws_security_group.demo[1] => secgp-web",
      - "aws_security_group.demo[2] => secgp-app",
      - "aws_security_group.demo[3] => secgp-db",
      + "aws_security_group.demo[1] => secgp-app",
      + "aws_security_group.demo[2] => secgp-db",
    ]
```

38. Verify the **new** resources in the AWS portal ...




Note the new IDs for secgp-db and secgp-app !

39. In **terraform.tfvars**, comment line 13 and uncomment line 14 so the list is empty. This will remove all existing security groups ahead of the next phase.

40. Apply the changes:

```
terraform apply --auto-approve
```

# Phase 4 – Stable keys with for_each

## What Is Changing Here?

Up to this point, resources have been created using count, which means Terraform identifies them by **numeric index** (e.g. [0], [1]). These indexes are tied to the **position of items in a list**, not what those items represent. This is why reordering or inserting values earlier in the list caused widespread and often unexpected changes.

In this phase, we change two things:

- The input variable moves from a **list** to a **map**
- The resource moves from using count to using for_each

This introduces a fundamental shift in how Terraform tracks resources.

Instead of:

- aws_security_group.demo[0]
- aws_security_group.demo[1]

Terraform will now use:

- aws_security_group.demo["secgp-web"]
- aws_security_group.demo["secgp-app"]

Resources are no longer identified by position, but by **stable keys**.

This means:

- Reordering input data no longer causes changes
- Adding new entries does not affect existing resources
- Terraform can track resources based on logical identity rather than list position

This is the key step that removes the instability seen in earlier phases and leads to a more predictable and maintainable design

41. In **variables.tf**, comment line 27 and uncomment lines 30. This re-types the variable security_groups from list(string) to map(string).
42. In **main.tf**, uncomment lines 39 and 55, and comment lines 57 and 73. This switches to a resource block that references the re-typed security_groups variables.
43. In **terraform.tfvars**, comment line 14 and uncomment line 17. This populates the re-typed security_groups variable with initial values.
44. Before running Terraform, consider what is about to change. The resources will no longer be addressed by numeric index.
45. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show replacement behaviour because the resource addressing model itself has changed.

    Previously, Terraform used addresses such as:

    ```
    aws_security_group.demo[0]
    aws_security_group.demo[1]
    ```

    The new block uses for_each with a map, so Terraform will now use key-based addresses such as:

    ```
    aws_security_group.demo["web"]
    aws_security_group.demo["app"]
    ```

    This is the critical shift! Terraform is no longer identifying resources by their position in an ordered list. It is identifying them by stable keys.

46. Apply the changes:

    ```
    terraform apply --auto-approve
    ```

47. Verify the new state addresses:

    ```
    terraform state list
    ```

    You should now see key-based addresses rather than numeric indexes.

    ```
    aws_default_security_group.default
    aws_security_group.demo["app"]
    aws_security_group.demo["web"]
    aws_subnet.demo
    aws_vpc.main
    ```

## Proving the stable design

48. In **terraform.tfvars,** comment line 17 and uncomment line 18 so that **db** is inserted between **web** and **app**.
49. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show 1 to add, 0 to change, 0 to destroy.

    The new resource will be aws_security_group.demo[db]. This will have no effect on the existing resources [web] and [app], because their keys are unchanged and are not tied to their positions in the list.

    ```
    terraform apply --auto-approve
    ```

50. In **terraform.tfvars**, comment line 18 and uncomment lines 19. This re-orders the values for the security_groups variable.
51. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show no changes.

    **Why?** Because although the order of the entries in the list has changed, the keys have not. Terraform is using those keys for identity, so reordering the map does not cause churn.

    This is the stable end state the lab is designed to lead you toward. The map provides explicit keys, and for_each uses those keys to maintain stable addressing in state.

    **Best-practice conclusion:** For long-lived infrastructure, prefer for_each with stable map keys and explicit structured values. Use simple ordered lists with care, because ordinal indexing is fragile.

# Phase 5 – Clean up

52. Destroy everything created in this lab:

    ```
    terraform destroy --auto-approve
    ```

****** Congratulations, you have completed this lab. ******